

EXAME ÉPOCA RECURSO (2ª PARTE, COM CONSULTA, DURAÇÃO: 75 min., 9 valores)

Nota: Nas perguntas seguintes, quando solicitado, deve usar troços de código Java essencial, podendo intermediar com linhas de comentário. Escrever código irrelevante com definições de *import* de *packages*, *static main*, *System.out.println*; *public* etc. é perda de tempo que lhe pode faltar para aspetos mais importantes.

9. [3 val.] Considere que existe numa empresa um conjunto de N servidores gRPC que suportam o acesso a diferentes bases de dados, cujo contrato é identificado do lado direito. Por forma a tornar transparente aos clientes esses servidores das diferentes bases de dados, foi decidido implementar um novo servidor gRPC que disponibiliza o serviço *FrontEndService* cujo contrato se apresenta do lado esquerdo. A função deste novo servidor é, por cada pedido dos clientes (*FEQuery*), realizar pedidos (*Query*) aos N servidores existentes, enviando os N resultados (*Result*) em *streaming* para o cliente.

```
// Front-end Contract
service FrontEndService {
    rpc queryFE(FEQuery) returns (stream FEResult);
}
message FEQuery {
    string feQueryID = 1; // Identifier
    string feQueryString = 2; // SQL command
}
message FEResult {
    string feQueryID = 1; // Query Identifier
    repeated string feRows = 2; // Result set
}
```

```
// Query servers Contract
service QueryService {
    rpc queryQS(Query) returns (Result);
}
message Query {
    string queryID=1; // Identifier
    string queryString=2; // SQL command
}
message Result {
    string queryID=1; // Query Identifier
    repeated string rows=2; // Result set
}
```

- a) [1.5 val.] Apresente o código Java essencial da classe que implementa o contrato no servidor gRPC que disponibiliza o serviço *FrontEndService*, sem ser necessário indicar o código do método *static main* de inicialização do servidor. Assuma que existe no servidor uma variável *static* com os pares (IP, porto) de todos os servidores de acesso às bases de dados.
- b) [1.5 val.] Apresente o código Java essencial da aplicação cliente que permite enviar em paralelo 5 pedidos ao servidor gRPC que disponibiliza o serviço *FrontEndService*.
10. [4 val.] Considere um cenário de uma empresa de vendas online que disponibiliza uma interface *publish/subscribe* baseada em RabbitMQ para que qualquer aplicação cliente possa submeter mensagens com pedidos de encomenda, confirmação de pagamentos e obtenção do estado das encomendas com as seguintes *routing keys*: "Order", "OrderPaid" e "Status". Internamente a empresa tem 3 serviços para tratar das encomendas: Um serviço de validação das encomendas, um serviço financeiro; e um serviço de entregas e notificações aos clientes. Essas notificações, sobre o estado das encomendas, são feitas pelo serviço de entregas com mensagens RabbitMQ.
- a) [2 val.] Apresente um diagrama que ilustre as interações e mensagens entre cliente e os serviços da empresa, identificando claramente: *exchanges* e *queues* necessárias, bem como as *routing* e *binding keys*;
- b) [2 val.] Apresente o código Java essencial de uma aplicação cliente que envia uma mensagem de pedido de estado de uma encomenda e obtém a respetiva mensagem de notificação.
11. [2 val.] [Não precisa apresentar código] Considere que tem de desenvolver um servidor gRPC que suporta um conjunto de operações num contrato a ser utilizado pelas aplicações cliente. Para garantir escalabilidade horizontal e tolerância a falhas será necessário que existam múltiplos servidores em execução. No entanto, cada cliente para se conectar a um servidor necessita de saber a sua localização, pelo que tem de existir um mecanismo para um cliente, antes de se conectar a um servidor, obter um par (IP, porto).

Apresentando um diagrama de interação entre as partes envolvidas e as mensagens necessárias, descreva as vantagens e desvantagens de usar comunicação por grupos com *multicast*, por exemplo *Spread Toolkit*, para que os clientes possam obter o par (IP, port) de um servidor. A solução deve garantir distribuição de carga, isto é, os múltiplos clientes devem conectar-se com equilíbrio pelos vários servidores.